

JSON

JavaScript Object Notation · syntaxe · modélisation · API · JSON Schema · jq · programmation

"titre" chasse fixe, gras : texte à écrire tel quel
fichier italique : valeur à remplacer

✓ valide · ✗ invalide

Maj+Alt+F touche ou combinaison de touches

❗ piège fréquent

QU'EST-CE QUE JSON ?

- **JSON** (*JavaScript Object Notation*) : format **texte** d'échange de données structurées, lisible par l'humain comme par la machine.
- Issu de la syntaxe de JavaScript, mais **indépendant de tout langage** : tous savent le lire et l'écrire.
- Normes : **RFC 8259** (IETF, 2017) et **ECMA-404** — même grammaire.
- Type de média **application/json** · extension **.json**.
- Encodage **UTF-8** obligatoire entre systèmes ; sans BOM (marque d'ordre des octets).

OÙ LE RENCONTRE-T-ON ?

- **API web** : requêtes et réponses des services en ligne.
- **Configuration** : **package.json**, **settings.json** de VS Code...
- **Stockage** : bases documentaires, colonnes **jsonb** de PostgreSQL, journaux, carnets Jupyter.

LES SIX TYPES DE VALEURS

objet	<code>{ "nom": "Eve" }</code>	clés → valeurs, sans ordre
tableau	<code>[1, "a", null]</code>	liste ordonnée
chaîne	<code>"texte"</code>	guillemets doubles, Unicode
nombre	<code>42 · -3.5 · 6.02e23</code>	un seul type numérique
booléen	<code>true · false</code>	minuscules, sans guillemets
nul	<code>null</code>	absence de valeur

Objets et tableaux **contiennent** d'autres valeurs et s'imbriquent librement. Ni date, ni binaire (page 2).

COMPACT OU INDENTÉ

```
["titre":"Candide","tags":["conte"]]  
  
{  
  "titre": "Candide",  
  "tags": [  
    "conte"  
  ]  
}
```

Même donnée : les espaces **entre** les éléments n'ont aucun sens. Compact pour transmettre, indenté pour lire.

REPÈRES

2000–2002	format décrit et diffusé par Douglas Crockford (site json.org)
2006	RFC 4627 : première spécification de l'IETF
2014	RFC 7159 : toute valeur admise à la racine
2017	RFC 8259 (norme Internet STD 90) et ECMA-404, 2 ^e édition : UTF-8 obligatoire entre systèmes

ANATOMIE D'UN DOCUMENT

```
{  
  "titre": "Candide",  
  "annee": 1759,  
  "disponible": true,  
  "prix": null,  
  "auteur": {  
    "nom": "Voltaire",  
    "naissance": 1694  
  },  
  "tags": ["conte", "philosophie"]  
}
```

- 1 **Objet** racine, entre accolades { }.
- 2 **Membre** : une **clé** (chaîne entre guillemets doubles), :, puis une **valeur** — ici une chaîne.
- 3 **Nombre** : sans guillemets. Une **virgule** sépare les membres.
- 4 Littéraux **true**, **false**, **null** : minuscules, sans guillemets.
- 5 Une valeur peut être un **objet imbriqué**...
- 6 Pas de virgule après le **dernier** membre.
- 7 ... ou un **tableau** : liste ordonnée entre crochets [].

LES DIX RÈGLES D'OR

1. Chaînes et clés entre **guillemets doubles** : **"nom"**, jamais **'nom'**.
 2. Toute clé est une chaîne : { **"a": 1** }, jamais { **a: 1** }.
 3. Une **virgule** **entre** deux éléments, jamais après le dernier.
 4. Deux-points entre la clé et sa valeur.
 5. **true**, **false**, **null** en minuscules, sans guillemets.
 6. Ni **undefined**, ni **NaN**, ni **Infinity**, ni fonction.
 7. Nombres en base 10, sans + initial ni zéro de tête.
 8. Dans une chaîne, échapper ", \ et les caractères de contrôle.
 9. **Aucun commentaire** : ni // ni /* */.
 10. Une seule valeur racine ; des clés uniques dans chaque objet.
- L'ordre des clés d'un objet ne porte aucun sens ;
l'ordre des éléments d'un tableau, si.

DÉBOGUEUR UN JSON INVALIDE

1. **Indenter** pour voir la structure : **jq · f**, **py -m json.tool f**, ou **Maj+Alt+F** dans VS Code.
 2. **Lire la position** donnée par le message : ligne, colonne ou caractère.
 3. Regarder **juste avant** : l'erreur réelle précède souvent l'endroit signalé.
 4. Vérifier dans l'ordre : virgules, guillemets, paires { } et [], littéraux.
 5. Fichier venu d'ailleurs : BOM en tête ? encodage autre qu'UTF-8 ?
- Messages d'erreur types : page 6.

ERREURS FRÉQUENTES

✗ FAUX	✓ CORRECT
<code>{ 'a': 1 }</code>	<code>{ "a": 1 }</code>
<code>{ a: 1 }</code>	<code>{ "a": 1 }</code>
<code>[1, 2, 3,]</code>	<code>[1, 2, 3]</code>
<code>{ "a": 1 "b": 2 }</code>	<code>{ "a": 1, "b": 2 }</code>
<code>{ "ok": True }</code>	<code>{ "ok": true }</code>
<code>{ "x": None }</code>	<code>{ "x": null }</code>
<code>{ "x": undefined }</code>	<code>{ "x": null }</code>
<code>{ "n": NaN }</code>	<code>{ "n": null }</code>
<code>{ "n": 007 }</code>	<code>{ "n": 7 }</code>
<code>{ "n": .5 }</code>	<code>{ "n": 0.5 }</code>
<code>{ "n": 1,5 }</code>	<code>{ "n": 1.5 }</code>
<code>"C:\dossier"</code>	<code>"C:\\dossier"</code>
<code>{ "a": 1 } { "b": 2 }</code>	<code>[{ "a": 1 }, { "b": 2 }]</code>
<code>{ "a": 1 } // note</code>	<code>{ "a": 1 }</code>

ATOUS ET LIMITES

ATOUS

- Grammaire minuscule : six types, quelques signes.
- Lisible à l'œil nu, facile à écrire à la main.
- Natif dans les navigateurs ; analyseurs dans tous les langages.
- Plus léger que XML pour la plupart des données.

LIMITES

- Ni commentaires, ni dates, ni binaire.
- Précision des nombres laissée aux analyseurs.
- Clés répétées dans chaque objet : volumineux en masse.
- Aucune validation intégrée : il faut un schéma (page 4).

VOCABULAIRE

membre	une paire " clé " : valeur dans un objet
clé · nom	la chaîne qui désigne une valeur
élément	une valeur dans un tableau
racine	la valeur qui englobe tout le document
sérialiser	données en mémoire → texte JSON (encoder)
désérialiser	texte JSON → données (analyser, <i>parser</i>)
analyser	programme qui lit et vérifie le JSON (<i>parser</i>)
indenter	mettre en forme avec retraits ; l'inverse : minifier
bien formé	conforme à la grammaire JSON
valide	conforme, en plus, à un schéma (page 4)

2 · Syntaxe en détail

OBJETS

<code>{}</code>	objet vide
<code>{"a": 1, "b": 2}</code>	deux membres séparés par une virgule
<code>{"prénom": "Anne"}</code>	clé libre : accents, espaces, symboles permis
<code>{}: 0}</code>	même la clé vide est permise
<code>{"a": 1, "a": 2}</code>	❗ clé en double : « devrait » être unique (RFC 8259)

Clés en double : la plupart des analyseurs gardent la **dernière** valeur (Python, JavaScript) ; d'autres signalent une erreur ou conservent toutes les paires. Résultat imprévisible : les proscrire.

Ordre des membres sans signification : `{"a":1,"b":2} = {"b":2,"a":1}`. Ne jamais s'y fier.

TABLEAUX

<code>[]</code>	tableau vide
<code>[1, 2, 3]</code>	éléments ordonnés : <code>[3, 2, 1]</code> est un autre tableau
<code>["a", 1, true, null]</code>	types mêlés : permis, mais à éviter
<code>[[1, 2], [3, 4]]</code>	tableaux imbriqués (matrice)
<code>[{"id": 1}, {"id": 2}]</code>	tableau d'objets : la forme d'une table de données

CHAÎNES ET ÉCHAPPEMENTS

<code>\"</code>	guillemet double
<code>\\</code>	barre oblique inverse
<code>\/</code>	barre oblique (facultatif : / suffit)
<code>\n · \r</code>	saut de ligne · retour chariot
<code>\t</code>	tabulation
<code>\b · \f</code>	retour arrière · saut de page
<code>\u00e9</code>	caractère Unicode en 4 chiffres hexadécimaux : é
<code>\ud83d\ude00</code>	au-delà de U+FFFF : paire de substitution (ici un émoji)

À échapper **obligatoirement** : le guillemet `"`, la barre oblique inverse `\` et les caractères de contrôle U+0000 à U+001F (saut de ligne, tabulation...).

❗ Invalides : `\' \x41 \0 \a`. Pas de retour à la ligne brut dans une chaîne : écrire `\n`.

En UTF-8, `"été"` et `"\u00e9t\u00e9"` sont la même chaîne.

Chemins Windows : `"C:\\Users\\docs"` — ou `"C:/Users/docs"`, accepté par la plupart des programmes.

ESPACES, RACINE, LIMITES

- Espaces permis **entre** les éléments : espace, tabulation, saut de ligne, retour chariot — et eux seuls.
- Retrait conventionnel : 2 ou 4 espaces. Une ligne vide finale est d'usage, non requise.
- La racine peut être **n'importe quelle valeur** : `42` ou `"ok"` forment un document JSON (RFC 8259) ; d'anciens analyseurs n'acceptent qu'un objet ou un tableau.
- Taille, profondeur, longueur des chaînes : chaque analyseur fixe ses propres limites.

TOUTE LA GRAMMAIRE

```
valeur = objet / tableau / chaîne / nombre
        / "true" / "false" / "null"
objet  = "{" [ membre *( "," membre ) ] "}"
membre = chaîne ":" valeur
tableau = "[" [ valeur *( "," valeur ) ] "]"
chaîne  = guillemet *caractère guillemet
nombre  = [ "-" ] entier [ fraction ] [ exposant ]
entier   = "0" / ( chiffre-1-à-9 *chiffre )
fraction = "." 1*chiffre
exposant = ( "e" / "E" ) [ "-" / "+" ] 1*chiffre
```

Notation ABNF simplifiée d'après RFC 8259 : / ou [] facultatif · * zéro, une ou plusieurs fois · 1* au moins une fois. Des espaces peuvent entourer chaque signe { } [] : , .

NOMBRES

-	12	.75	e+3
SIGNE FACULTATIF	PARTIE ENTIÈRE	FRACTION FACULTATIVE	EXPOSANT FACULTATIF
✓ VALIDES	<code>0</code> <code>-0</code> <code>42</code> <code>3.14</code> <code>-0.5</code> <code>1e10</code> <code>1E-3</code> <code>2.5e+8</code>		
✗ INVALIDES	<code>+1</code> <code>01</code> <code>.5</code> <code>5.</code> <code>1.e3</code> <code>0x1F</code> <code>1_000</code> <code>1,5</code>		
	<code>NaN</code> <code>Infinity</code>		

Un seul type numérique : `1`, `1.0` et `1e0` expriment la même quantité ; certains langages les distinguent (Python : `int` et `float`).

PRÉCISION

- La norme ne fixe aucune limite ; la plupart des analyseurs utilisent des **doubles IEEE 754** (64 bits).
- Entiers exacts garantis de $-(2^{53}-1)$ à $2^{53}-1 = 9007199254740991$ (RFC 8259, § 6).
- ❗ Au-delà, arrondi **silencieux** : en JavaScript, `9007199254740993` devient `9007199254740992`.
- Réels : arrondi binaire — `0.1 + 0.2` donne `0.30000000000000004`.
- **Identifiants** longs en chaîne ; **montants** en centimes entiers ou en chaîne décimale : `"19.90"`.

CÉ QUE JSON NE SAIT PAS REPRÉSENTER

BESOIN	CONVENTION USUELLE
date, heure	chaîne ISO 8601 : <code>"2026-09-29T14:30:00Z"</code>
durée	chaîne ISO 8601 <code>"PT1H30M"</code> , ou nombre + unité dans le nom : <code>"dureeSec": 5400</code>
binaire (image...)	chaîne Base64, ou lien vers le fichier
commentaire	clé dédiée <code>"_note"</code> ; ou format JSONC / JSON5 (page 3)
undefined	omettre la clé, ou <code>null</code>
grand entier, décimal exact	chaîne : <code>"12345678901234567890"</code>
référence, cycle	identifiant : <code>"auteurId": "a-42"</code>
type d'objet	champ discriminant : <code>"type": "livre"</code>

JSON N'EST PAS JAVASCRIPT

JAVASCRIPT ACCEPTE	JSON EXIGE
<code>{nom: 'Candide'}</code>	<code>{"nom": "Candide"}</code>
<code>[1, 2,]</code>	<code>[1, 2]</code>
<code>// commentaire</code>	— (aucun commentaire)
<code>undefined · NaN · Infinity</code>	<code>null</code> , ou une chaîne
<code>new Date()</code>	<code>"2026-09-29T14:30:00Z"</code>

❗ Ne jamais lire du JSON avec `eval()` : exécution de code arbitraire. Utiliser `JSON.parse()`.

UNICODE ET ENCODAGE

- Texte JSON échangé : **UTF-8**, sans BOM ; un analyseur **peut** ignorer un BOM reçu, sans y être tenu.
- Tout caractère Unicode peut figurer tel quel dans une chaîne, sauf `"`, `\` et les caractères de contrôle.
- ❗ Même aspect, octets différents : « é » s'écrit en un point de code (U+00E9) ou en deux (e + accent combinant U+0301). Normaliser (NFC) avant de comparer clés et valeurs.
- Longueur d'une chaîne : un émoji compte 2 en JavaScript (UTF-16), 1 en Python.

COMPARER DEUX DOCUMENTS

- Même donnée, textes différents : ordre des clés, espaces, `1.0` et `1`, `"\u00e9"` et `"é"`.
- Comparer les **données**, pas le texte : `jq -S . a.json > a2.json`, idem pour `b`, puis `diff a2.json b2.json`.
- Signature, empreinte (hachage) : sérialisation canonique **JCS** (RFC 8785) — clés triées, forme des nombres et des chaînes fixée.

3 · Modéliser et échanger

EXEMPLE : UNE RESSOURCE BIEN CONÇUE

```
{
  "id": "cmd-2026-00042",
  "statut": "expediee",
  "creeLe": "2026-09-29T14:30:00Z",
  "client": { "id": "c-1187", "nom": "Exemple SARL" },
  "lignes": [
    { "sku": "LIV-001", "quantite": 2, "prix": "12.50" },
    { "sku": "LIV-007", "quantite": 1, "prix": "8.00" }
  ],
  "total": { "montant": "33.00", "devise": "EUR" },
  "remarque": null
}
```

Identifiants en chaînes · statut lisible · date ISO avec fuseau · montants exacts avec devise · clé **remarque** présente mais vide.

RÈGLES DE CONCEPTION

- Un seul style de nommage : **camelCase** (**dateCreation**) ou **snake_case** (**date_creation**) ; clés sans accent ni espace.
- Un champ, un type : **quantite** est toujours un nombre, jamais tantôt 2, tantôt "2".
- Listes : nom au pluriel ; tableau vide [] plutôt que **null**.
- Booléens nommés comme une question : **estActif**, **enStock**.
- Unité dans le nom : **poidsKg**, **dureeSec**, **tailleOctets**.
- Énumérations en chaînes lisibles : **"statut": "expediee"** plutôt que **"statut": 3**.
- Dates ISO 8601 avec fuseau : **Z** (UTC) ou **+02:00**.
- Montants : valeur exacte et code de devise ISO 4217 (**EUR**, **USD**).
- Imbrication modérée : au-delà de trois ou quatre niveaux, aplatir ou référencer.
- ⓘ Clé **absente** (non fournie) et valeur **null** (vide) ne disent pas la même chose : fixer la convention et la documenter.

OBJET OU TABLEAU ?

une chose et ses propriétés	objet : { "titre": ..., "annee": ... }
une liste ordonnée	tableau : [...]
des enregistrements	tableau d'objets : [{...}, {...}]
un accès par clé	objet indexé : { "fr": "Bonjour", "en": "Hello" }
un couple de taille fixe	tableau court, ordre documenté : [2.35, 48.85]

ⓘ GeoJSON (RFC 7946) écrit les positions [**longitude**, **latitude**] : l'inverse de l'usage courant.

LA FAMILLE JSON

JSON Lines	un document par ligne (.jsonl, dit aussi NDJSON) : journaux, gros volumes, flux
JSONC	JSON + commentaires : settings.json de VS Code
JSON5	+ commentaires, virgules finales, guillemets simples, clés nues
GeoJSON	lieux et géométries (RFC 7946)
JSON-LD	données liées (W3C) ; balisage schema.org des pages web
I-JSON	profil strict pour l'interopérabilité (RFC 7493)
BSON · CBOR · MessagePack	équivalents binaires, plus compacts

FAIRE ÉVOLUER UN FORMAT

- Listes : envelopper le tableau dans un objet — { "donnees": [...], "suivant": ... } — pour ajouter plus tard pagination et métadonnées sans rien casser.
- Ajouter un champ facultatif : sans risque. Retirer, renommer ou changer le type d'un champ : rupture → nouvelle version.
- Clients tolérants : ignorer les champs inconnus au lieu d'échouer.

JSON SUR HTTP (API WEB)

Content-Type: application/json	le corps envoyé est du JSON
Accept: application/json	le client demande du JSON en réponse

CODES DE RÉPONSE UTILES

200 · 201	succès · ressource créée (en-tête Location)
204	succès sans corps : rien à analyser
400	requête mal formée — par exemple JSON invalide
401 · 403	non authentifié · non autorisé
404 · 409	ressource introuvable · conflit avec l'état actuel
415	type de média refusé : vérifier Content-Type
422	JSON bien formé, mais données refusées
429 · 500	trop de requêtes : ralentir · erreur du serveur

charset=utf-8 est superflu : JSON est en UTF-8 et RFC 8259 ne définit aucun paramètre **charset**.

DÉCRIRE UNE ERREUR : RFC 9457

```
{
  "type": "https://example.com/erreurs/stock",
  "title": "Stock insuffisant",
  "status": 409,
  "detail": "Il reste 1 exemplaire de LIV-007.",
  "instance": "/commandes/cmd-2026-00042"
}
```

type	URI de la catégorie d'erreur (par défaut : about:blank)
title	résumé lisible, identique pour toute la catégorie
status	code HTTP
detail · instance	explication · URI propres à cette occurrence

Type de média : **application/problem+json**. Des membres supplémentaires sont permis.

DÉSIGNER ET MODIFIER UNE PARTIE

JSON POINTER (RFC 6901)

""	le document entier
/lignes/0/sku	clé lignes , élément 0, clé sku → "LIV-001"
~0 · ~1	~ et / à l'intérieur d'un nom de clé

JSON PATCH (RFC 6902) · application/json-patch+json

```
[
  { "op": "test", "path": "/statut", "value": "expediee" },
  { "op": "replace", "path": "/statut", "value": "livree" },
  { "op": "add", "path": "/lignes/-",
    "value": { "sku": "LIV-009", "quantite": 1 } },
  { "op": "remove", "path": "/remarque" }
]
```

Opérations : **add remove replace move copy test**, appliquées dans l'ordre ; si l'une échoue, le patch entier échoue. **/lignes/-** : après le dernier élément.

JSON MERGE PATCH (RFC 7396) · application/merge-patch+json

On envoie un objet partiel : ses membres remplacent ceux de la cible, **null** **supprime** la clé, un tableau est remplacé en entier. Ex. { "statut": "livree", "remarque": null }.

JSON FACE AUX AUTRES FORMATS

FORMAT	FORCES	LIMITES
JSON	simple, universel, natif du web	ni commentaires ni dates
XML	attributs, espaces de noms, documents	verbeux, lourd à traiter
YAML	très lisible, commentaires ; lit le JSON (YAML 1.2)	indentation significative, typage piègeux
TOML	configuration claire, dates natives	peu fait pour l'imbrication profonde
CSV	tableaux plats, tableurs	ni types ni imbrication

4 · JSON Schema : décrire et valider

À QUOI SERT UN SCHÉMA ?

- **Décrire** la structure attendue : types, champs obligatoires, valeurs permises.
- **Valider** automatiquement données reçues, fichiers de configuration, réponses d'API.
- **Documenter**, générer formulaires, code et tests ; OpenAPI 3.1 décrit les données des API en JSON Schema.
- Un schéma est lui-même un document JSON. Version en vigueur : **2020-12** (précédente : 2019-09).

EXEMPLE : LE SCHÉMA DE LA COMMANDE (PAGE 3)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.com/schemas/commande.json",
  "title": "Commande",
  "type": "object",
  "required": ["id", "statut", "lignes"],
  "properties": {
    "id": { "type": "string", "pattern": "^cmd-\\d{4}-\\d{5}$" },
    "statut": {
      "enum": ["en-attente", "expediee", "livree", "annulee"]
    },
    "creeLe": { "type": "string", "format": "date-time" },
    "lignes": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "#/$defs/ligne" }
    },
    "remarque": { "type": ["string", "null"], "maxLength": 500 }
  },
  "$defs": {
    "ligne": {
      "type": "object",
      "required": ["sku", "quantite"],
      "properties": {
        "sku": { "type": "string", "minLength": 1 },
        "quantite": { "type": "integer", "minimum": 1 },
        "prix": { "$ref": "#/$defs/decimal" }
      },
      "additionalProperties": false
    },
    "decimal": { "type": "string", "pattern": "^\\d+\\.\\d{2}$" }
  }
}
```

Dans une chaîne JSON, la barre oblique inverse d'une expression régulière se double : `\\d` pour `\\d`.

CE QUE LE SCHÉMA ACCEPTE OU REFUSE

DONNÉE	RÉSULTAT
la commande de la page 3	✓ valide
"quantite": 0	✗ minimum : 0 < 1
"quantite": 2.5	✗ type : pas un entier
"statut": "Livree"	✗ enum : la casse compte
"lignes": []	✗ minItems
ligne avec "couleur": "rouge"	✗ additionalProperties
sans "id"	✗ required
"prix": "12.5"	✗ pattern : deux décimales
"creeLe": "29/09/2026"	✓ en général ! format n'est qu'une annotation
clé inconnue "urgent": true à la racine	✓ racine sans additionalProperties: false

OUTILS

ajv	validateur JavaScript (Node.js), le plus répandu
jsonschema	Python : validate(instance=doc, schema=s)
check-jsonschema	ligne de commande, intégration continue
VS Code	valide pendant la saisie si le fichier déclare "\$schema", ou via le réglage json.schemas
SchemaStore.org	catalogue de schémas des fichiers de configuration courants

MOTS-CLÉS ESSENTIELS

TOUS TYPES

type	"string" "number" "integer" "boolean" "object" "array" "null" — ou une liste : ["string", "null"]
enum · const	valeurs permises · valeur imposée
title · description · default	documentation ; ! default ne remplit aucune valeur

CHAÎNES

minLength · maxLength	longueur en caractères
pattern	expression régulière (ECMA-262), non ancrée
format	date-time date time duration email uri uuid ipv4 ipv6 hostname...

NOMBRES

minimum · maximum	bornes incluses
exclusiveMinimum	borne exclue (un nombre) ; idem exclusiveMaximum
multipleOf	multiple de : 0.01, 5...

OBJETS

properties	schéma de chaque clé déclarée
required	liste des clés obligatoires
additionalProperties	clés non déclarées ; false : interdites
patternProperties	schéma des clés qui suivent un motif
minProperties · maxProperties	nombre de clés ; propertyName : contraintes sur leurs noms
dependentRequired	si la clé A est présente, B devient obligatoire
unevaluatedProperties	comme additionalProperties, mais voit à travers allOf, \$ref...

TABLEAUX

items	schéma de chaque élément
prefixItems	schémas des premiers éléments, rang par rang (n-uplet)
minItems · maxItems	nombre d'éléments
uniqueItems	true : aucun doublon
contains	au moins un élément conforme (minContains, maxContains)

COMBINER ET RÉUTILISER

allOf	conforme à tous les sous-schémas
anyOf	conforme à au moins un
oneOf	conforme à exactement un
not	non conforme
if · then · else	validation conditionnelle
\$defs · \$ref	définitions réutilisables : "\$ref": "#/\$defs/ligne" (JSON Pointer)
\$schema · \$id	version du langage · identifiant du schéma

PIÈGES

- Sans **additionalProperties: false**, une clé mal orthographiée ("quantitee") passe inaperçue.
- **required** se place dans l'objet **parent**, pas dans la propriété.
- **pattern** non ancré : "[0-9]+" accepte "abc1" → écrire "^ [0-9]+\$".
- **format** n'est qu'une **annotation** par défaut : activer son contrôle dans le validateur.
- **integer** accepte **1.0** : un nombre sans partie décimale est un entier.
- **oneOf** échoue si **deux** branches conviennent : **anyOf** suffit souvent.
- Anciens schémas (draft-04, draft-07) : **definitions** → **\$defs** ; **items** en liste → **prefixItems** ; **exclusiveMinimum** booléen → nombre.

5 · jq et autres outils

JQ : LIRE ET FILTRER

jq **-r** **' .livres[0].titre '** **livres.json**

COMMANDE

OPTIONS

FILTRE

FICHIER

Sans fichier, jq lit l'entrée standard. | enchaîne les filtres, comme dans le shell.

FICHIER D'EXEMPLE : LIVRES.JSON

```
{ "livres": [
  { "titre": "Candide", "auteur": "Voltaire",
    "annee": 1759, "tags": ["conte", "philosophie"] },
  { "titre": "Notre-Dame de Paris", "auteur": "Hugo",
    "annee": 1831, "tags": ["roman"] },
  { "titre": "Les Misérables", "auteur": "Hugo",
    "annee": 1862, "tags": ["roman"] }
]
```

.	tout le document, indenté et coloré
.livres[0].titre	"Candide"
.livres[-1].annee	1862 (dernier élément)
.livres[.].titre	les trois titres, un par ligne
.livres length	3
.livres[0] keys	["annee", "auteur", ...] (triées)
.livres[0].prix // "inconnu"	clé absente → null, sans erreur ; // : valeur de repli
.livres[] select(.auteur == "Hugo") .titre	les deux titres de Hugo
.livres[] select(any(.tags[]; . == "roman"))	les romans
.livres map(.titre)	["Candide", ...] : un tableau
[.livres[].annee] add / length	1817.33... : la moyenne
.livres group_by(.auteur) map({auteur: .[0].auteur, n: length})	[{"auteur": "Hugo", "n": 2}, ...]
.. .titre? // empty	toutes les clés titre, à toute profondeur
.livres[] "\(.titre) (\(.annee))"	avec -r : Candide (1759)...

JQ : FONCTIONS UTILES

if .annee < 1800 then "ancien" else "moderne" end	condition
select(.titre test("^les"; "i"))	expression régulière (i : sans casse)
ascii_lowercase · split(" ") · join(", ")	traiter les chaînes
unique · flatten · min_by(f) · add	traiter les tableaux

RECETTES

jq . brut.json > joli.json	indenter (-c : compacter)
jq empty f.json && echo "valide"	valider (code de retour)
jq -c ' .[] ' t.json > f.jsonl	tableau → JSON Lines
jq -s . f.jsonl	JSON Lines → tableau
jq -r ' .[] [.a, .b] @csv ' f.json	exporter en CSV
jq -s ' .[0] * .[1] ' a.json b.json	fusion profonde de deux objets

❗ Sous Windows PowerShell 5.1, les guillemets doubles d'un filtre sont mal transmis : mettre le filtre dans un fichier (jq -f) ou utiliser Git Bash.

GROS VOLUMES

- Traiter **en flux**, sans tout charger en mémoire : JSON Lines, jq --stream, ijson (Python).
- Réduire le volume : gzip (très efficace sur JSON) ou format binaire (CBOR, MessagePack).

JQ : OPTIONS

-r	chaînes brutes, sans guillemets (pour les scripts)
-c	sortie compacte : un document par ligne
-s	réunir toutes les entrées dans un tableau (slurp)
-n	ne rien lire : construire du JSON de toutes pièces
-S	trier les clés
-e	code de retour selon le résultat (false, null → 1)
-R	lire du texte brut : chaque ligne devient une chaîne
-f fichier	lire le filtre dans un fichier
--arg n v	variable \$n : une chaîne
--argjson n v	variable \$n : une valeur JSON
--indent n · --tab	retrait de n espaces · tabulations

JQ : TRANSFORMER

.livres[0].prix = 12	ajouter ou remplacer une valeur
.livres[].annee = tostring	modifier sur place
del(.livres[].tags)	supprimer une clé partout
.livres = sort_by(.titre)	trier le tableau en place
.livres[] {titre, annee}	ne garder que certaines clés
with_entries(.key = ascii_upcase)	renommer les clés
to_entries · from_entries	objet ↔ [{"key": ..., "value": ...}]
.livres[] [.titre, .annee] @csv	ligne CSV (avec -r)
@tsv · @base64 · @uri · @html	autres encodages de sortie

AUTRES OUTILS

py -m json.tool f	valider et indenter (Python) ; python3 sous Linux ; -m json depuis Python 3.14
curl --json '{"a": 1}'	POST en JSON, en-têtes Content-Type et Accept posés (curl ≥ 7.82) ; --json @f.json : depuis un fichier
curl -s url jq '.data[].id'	interroger une API et filtrer
Maj+Alt+F	VS Code : formater ; validation par schéma, repli des blocs
Firefox	visionneuse JSON intégrée : replier, filtrer, JSON brut
gron	aplatit en lignes json.livres[0].titre = "Candide"; → grep

Options de json.tool : --sort-keys, --compact, --no-ensure-ascii, --json-lines.

POWERSHELL

\$d = Get-Content f.json -Raw ConvertFrom-Json	fichier → objets
\$d.livres[0].titre	naviguer avec des points
\$d ConvertTo-Json -Depth 10	objets → JSON
Invoke-RestMethod url	appel d'API : la réponse JSON devient un objet
ConvertFrom-Json -AsHashtable · Test-Json	PowerShell 7 (absents de la 5.1)

❗ ConvertTo-Json s'arrête à la profondeur 2 par défaut : plus bas, les objets deviennent du texte ("System.Collections.Hashtable") → toujours préciser -Depth.

❗ @(1) | ConvertTo-Json donne 1 : le pipeline déroule le tableau.

ConvertTo-Json -InputObject @(1) donne [1].

❗ Windows PowerShell 5.1 : curl y désigne Invoke-WebRequest → taper curl.exe.

6 · Programmer avec JSON

JAVASCRIPT

```
const obj = JSON.parse(texte); // texte → valeur
const txt = JSON.stringify(obj); // valeur → texte
JSON.stringify(obj, null, 2); // indenté
JSON.stringify(obj, ["id", "nom"]); // ces clés seules
const d = JSON.parse(texte, (cle, val) => // « reviver »
  cle === "creele" ? new Date(val) : val);
const rep = await fetch(url); // API web
const data = await rep.json();
```

undefined · fonction · clé omise ; dans un tableau : null
Symbol

NaN · Infinity null
Date chaîne ISO :
"2026-09-29T14:30:00.000Z"

Map · Set {} : contenu perdu
-0 0

BigInt · objet erreur **TypeError**
cyclique

toJSON() si l'objet la définit, sa valeur de retour est
sérialisée

Ce que devient chaque valeur avec **JSON.stringify** (ci-dessus). Au retour, **JSON.parse** rend les dates en chaînes (les convertir avec un *reviver*) et arrondit les entiers au-delà de 2^{53} .

PYTHON

```
import json

with open("f.json", encoding="utf-8") as f:
    data = json.load(f) # fichier → objets

texte = json.dumps(data, ensure_ascii=False, indent=2)
data = json.loads(texte) # chaîne → objets

with open("sortie.json", "w", encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False, indent=2)
```

JSON	PYTHON
objet	dict
tableau	list (un tuple devient aussi un tableau)
chaîne	str
nombre entier · à virgule	int · float
true · false · null	True · False · None

- **ensure_ascii=True** par défaut : é s'écrit `\u00e9` → **ensure_ascii=False**.
- **NaN** et **Infinity** acceptés et produits par défaut, hors norme → **allow_nan=False**.
- Clés converties en chaînes : `{1: "a"} → {"1": "a"}` ; l'aller-retour ne rend pas l'original.
- **datetime**, **Decimal**, **set** → **TypeError : default=str**, ou conversion explicite.
- **parse_float=Decimal** : décimaux exacts. **sort_keys=True** : sortie stable, comparable.
- Sous Windows, toujours **encoding="utf-8"** ; fichier avec BOM : **"utf-8-sig"**.

DATES ET HEURES

"2026-09-29"	date seule (ISO 8601)
"2026-09-29T14:30:00Z"	instant en UTC : la forme la plus sûre
"2026-09-29T16:30:00+02:00"	même instant, avec décalage
1790692200	même instant en secondes Unix (JavaScript compte en millisecondes)

❗ Sans fuseau, **"2026-09-29T14:30:00"** est ambiguë : JavaScript la lit en heure locale.

AUTRES LANGAGES

LANGAGE	LIRE · ÉCRIRE
PHP	<code>json_decode(\$s, true) · json_encode(\$v)</code>
Java (Jackson)	<code>mapper.readValue(s, T.class) · mapper.writeValueAsString(v)</code>
C# (.NET)	<code>JsonSerializer.Deserialize<T>(s) · JsonSerializer.Serialize(v)</code>
Go	<code>json.Unmarshal(b, &v) · json.Marshal(v)</code>
Rust (serde_json)	<code>serde_json::from_str(s) · serde_json::to_string(&v)</code>
PowerShell	<code>ConvertFrom-Json · ConvertTo-Json -Depth 10</code>
PostgreSQL	<code>doc->>'titre' (texte) · jsonb_build_object(...)</code>
SQLite	<code>json_extract(doc, '\$.titre') · json_object(...)</code>

PHP : options **JSON_PRETTY_PRINT**, **JSON_UNESCAPED_UNICODE**, **JSON_THROW_ON_ERROR** (à combiner par `|`). Go : seuls les champs exportés (majuscule) sont sérialisés ; nom JSON fixé par la balise **json:"titre"**.

MESSAGES D'ERREUR COURANTS

PYTHON 3.14 · NODE.JS 24	CAUSE PROBABLE
Expecting property name enclosed in double quotes <i>Expected property name or '}'</i>	clé sans guillemets, ou entre guillemets simples
Illegal trailing comma before end of object <i>Expected double-quoted property name</i>	virgule finale
Expecting ',' delimiter <i>Expected ',' or '}' after property value</i>	virgule manquante, ou nombre mal écrit (007 , 1.)
Expecting value <i>Unexpected token 'T', '{"ok": True}' is not valid JSON</i>	valeur invalide : True , None , NaN (JS), 'x' , .5...
Expecting value: line 1 column 1 (char 0) <i>Unexpected token '<', '<!DOCTYPE ...</i>	réponse vide, ou page HTML reçue au lieu de JSON : vérifier le statut HTTP
Invalid \escape <i>Bad escaped character in JSON</i>	<code>\</code> isolé : chemin C:\dossier
Invalid control character at <i>Bad control character in string literal</i>	retour à la ligne brut dans une chaîne
Extra data <i>Unexpected non-whitespace character after JSON</i>	texte après la fin : documents collés, } en trop
Unexpected UTF-8 BOM (decode using utf-8-sig) <i>Unexpected token ',', ...</i>	BOM (invisible) en tête du fichier
– <i>Unexpected end of JSON input</i>	texte vide ou tronqué (réponse 204...)

Messages suivis de la position : **line 1 column 9 (char 8)** · **at position 8 (line 1 column 9)**. Les libellés varient selon les versions.

SÉCURITÉ

- Données reçues = **non fiables** : valider types, bornes et structure (schéma) avant usage.
- Jamais **eval()** pour lire du JSON : **JSON.parse**, **json.loads**.
- ❗ Ne jamais fabriquer du JSON par **concaténation** de chaînes : un guillemet dans une valeur casse ou détourne le document. Toujours passer par le sérialiseur.
- Limiter taille et profondeur : un document démesuré ou très imbriqué épuise mémoire ou pile (Python : **RecursionError**).
- Clés en double : deux analyseurs peuvent retenir des valeurs différentes et contourner un contrôle → les refuser.
- JavaScript : fusion récursive d'objets reçus → **pollution de prototype** par les clés **__proto__**, **constructor**, **prototype**.
- JSON inséré dans une page HTML : échapper `<` en `\u003c` pour neutraliser `</script>`.
- Aucun secret (mot de passe, clé d'API, jeton) dans un JSON versionné : variables d'environnement ou coffre-fort.